

Contents

SIREN, Explained From Scratch	1
Table of contents	1
1. The situation	2
2. The one idea you need: braking power	2
3. Building the maths, gently	3
4. The failure condition	5
5. Two ways to attack	7
6. Four kinds of attacker	8
7. White box — you know everything	8
8. Grey box — you know the kind of filter, not its setting	9
9. Weak black box — you know the danger formula is one of two	10
10. Strict black box — you know nothing about the filter	13
11. The ladder: climbing from black to white	15
12. Summary	15
 Appendix A — The mathematics in full	 16
A.0 Notation, and one clash to watch	17
A.1 The system model	17
A.2 The safety index ϕ	18
A.3 Where $L_f \phi$ and $L_g \phi$ come from	18
A.4 The sensitivity in closed form	19
A.5 Control authority c , derived	19
A.6 The demand	20
A.7 The margin g , and the feasibility theorem	20
A.8 Several constraints at once	21
A.9 Why the index-free ruler works	22
A.10 The attack objectives, in symbols	22
A.11 Where each quantity is computed	23
A.12 What this appendix does not establish	23

SIREN, Explained From Scratch

How to make a safe robot unsafe, using nothing but polite requests

This document explains the whole idea behind SIREN — the maths, where it comes from, and exactly what an attacker does in each of four situations. It assumes no calculus. Every symbol is defined the first time it appears, and every piece of jargon is either explained or avoided.

Table of contents

1. [The situation](#)
2. [The one idea you need: braking power](#)
3. [Building the maths, gently](#)
4. [The failure condition](#)

5. [Two ways to attack](#)
 6. [Four kinds of attacker](#)
 7. [White box](#)
 8. [Grey box](#)
 9. [Weak black box](#)
 10. [Strict black box](#)
 11. [The ladder: climbing from black to white](#)
 12. [Summary](#)
 13. [Appendix A — The mathematics in full](#)
-

1. The situation

A humanoid robot works in a room with people and obstacles. To stop it hurting anyone, it runs a **safety filter**: a piece of software that sits between “what the robot wants to do” and “what the robot actually does”.

a goal		the ordinary		the safety		the robot
("fetch	→	controller	→	filter	→	moves
the cup")		"move toward it"		"...but not		
				into that		
				person"		

Every step, the ordinary controller proposes a movement. The safety filter checks it. If the movement is dangerous, the filter replaces it with the closest safe alternative. This is a well-established design, and it works — **as long as a safe alternative exists**.

SIREN is a tool that automatically finds situations where no safe alternative exists. And it does so using only legitimate goals — the kind of request any ordinary user is allowed to make. No hacking, no fake sensor readings, no tampering with the robot. Just asking it to go somewhere.

That is what makes this a security problem and not merely an engineering limitation: the attacker never does anything they are not permitted to do.

2. The one idea you need: braking power

Imagine a car heading toward a wall.

The car’s **braking power** is how quickly it can kill its speed. On dry asphalt it is large. On ice it is tiny. There is a point — depending on speed, distance and grip — beyond which **no amount of pedal pressure stops the car in time**. The driver is doing everything right; the physics has simply run out.

A robot arm has exactly the same property. Call it **control authority**:

Control authority = how fast the robot’s motors can push danger away, right now, in the pose it is currently in.

(“Control authority” is a genuine, long-standing term in control engineering — pilots talk about losing pitch control authority. The specific way we measure it below is our own formulation.)

Two things determine it:

1. **Motor limits.** A joint can only move so fast.
2. **Geometry.** Whether moving those joints actually helps. If the arm is stretched out awkwardly and the obstacle sits right against its shoulder, the joints that could help may barely move the endangered part at all.

Control authority is **low** in awkward poses — arm extended, obstacle tucked close to a joint that cannot retreat. We call such a region a **low-authority basin**. It is a physical property of the robot and the room. It has nothing to do with which safety filter is installed.

The entire attack is: get the robot into a low-authority basin.

Once it is there, the safety filter is asked to do something the motors cannot physically do. No software can conjure power that does not exist.

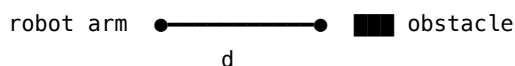
3. Building the maths, gently

Now let us make “braking power” precise. Five small steps.

Every quantity introduced here is derived properly in [Appendix A](#) — where φ , $\mathcal{L}_f \varphi$ and $\mathcal{L}_g \varphi$ come from, the closed form of the sensitivity, and the proof of the feasibility condition. This section is the readable version; the appendix is the reference one.

Step 1 — Clearance

Clearance is simply the distance between a part of the robot and an obstacle. We write it d .



Large d = safe. Small d = dangerous. d below zero would mean they overlap.

Step 2 — The danger number

The filter does not think in clearance directly; it uses a **danger number**, traditionally called the safety index. We write it with the Greek letter phi: φ . The simplest and most common form is

$$\varphi = d_{\min} - d$$

where $d_{\min}$ is the safety margin the designer chose — say 5 cm. Read it as “how far inside my comfort zone is this obstacle?”

$d > d_{min}$	$\rightarrow$	$\varphi < 0$	comfortably clear
$d = d_{min}$	$\rightarrow$	$\varphi = 0$	exactly on the boundary
$d < d_{min}$	$\rightarrow$	$\varphi > 0$	inside the comfort zone – the filter engages

So **the filter wakes up when ϕ becomes positive**. Its job from then on is to push ϕ back down.

There is a second common form of the danger number that also counts how fast the obstacle is being approached. We will come back to it in [section 9](#) — telling the two apart turns out to be an attack step in its own right.

Step 3 — How fast the danger number changes

We need to talk about rates: how much something changes per second. The standard notation is a dot on top, so $\dot{\phi}$ (say “phi-dot”) means “how fast the danger number is changing”.

- $\dot{\phi}$ negative $\rightarrow$ danger is falling $\rightarrow$ good, we are getting safer.
- $\dot{\phi}$ positive $\rightarrow$ danger is rising $\rightarrow$ bad.

The danger number changes for two separate reasons:

$$\dot{\phi} = (\text{what happens on its own}) + (\text{what our motors do})$$

The first part is called the **drift**. It is what the danger would do with all motors switched off — because the robot still has momentum, or because the obstacle (a person!) is moving toward it. We write it $L_f \phi$. You can read that symbol as simply “the drift of the danger number”.

In this project’s simulations the robot is controlled by velocity commands and the obstacles are stationary, so switching the motors off freezes everything: **the drift is zero**. That simplifies a lot, and we will flag where it matters.

The second part is what we control. Each joint of the robot, when it moves, changes the danger number by some amount. That per-joint amount is a **sensitivity** — think of it as a gear ratio: “if I turn joint 3 at one unit of speed, the danger number changes by this much.”

The whole list of sensitivities, one per joint, is written $L_g \phi$. Read it as “the sensitivity list of the danger number”. If joint k has sensitivity $[L_{gphi}]_k$ and we run it at speed u_k , then

$$\text{what our motors do} = [L_{gphi}]_1 \cdot u_1 + [L_{gphi}]_2 \cdot u_2 + \dots + [L_{gphi}]_n \cdot u_n$$

Multiply each joint’s sensitivity by its speed, and add up. Putting it together:

$$\dot{\phi} = \underbrace{L_f \phi}_{\text{drift}} + \underbrace{\sum [L_{gphi}]_k \cdot u_k}_{\text{what we control}}$$

That is the only equation you need to carry forward. (The symbol Σ , capital sigma, just means “add up over all the joints”.)

Step 4 — Control authority, precisely

Now we can answer: what is the fastest we could possibly push the danger down?

Each joint has a speed limit — call it u_{lim_k} — and can move **either direction**. So to push danger down as hard as possible, we run every joint at full speed in whichever direction helps. Joint k then contributes

$$u_{lim_k} \times |[L_{gphi}]_k|$$

The vertical bars mean absolute value (drop the minus sign) — because a joint that would increase danger when run forwards will decrease it when run backwards, and either way it contributes its full amount.

Add them all up and you have the control authority:

$$c = \sum \frac{u_{\text{lim}_k}}{\text{speed limit}} \times \frac{|[L_{\text{gphi}}]_k|}{\text{how much this joint moves the danger}}$$

c is the braking power. It is a single number, it depends only on the robot’s pose and the obstacle’s position, and — importantly — **it does not depend on which safety filter is installed.** It is physics, not software.

A useful special case: if a joint has **zero** sensitivity, it contributes nothing. If every joint has near-zero sensitivity for a given obstacle, then $c \approx 0$: the robot is in a pose where it simply cannot retreat. That is the bottom of a low-authority basin.

Step 5 — The demand

The filter does not merely want danger to fall; it insists on a **rate**. It demands

$$\dot{\phi} \leq -(\text{demand})$$

“danger must fall at least this fast.” The demand is a positive number, and there are exactly **two kinds** in use across the whole family of filters:

Kind	Formula	Behaviour
Constant	$\text{demand} = \eta$ (eta)	The same at all times, including right at the boundary
Proportional	$\text{demand} = \lambda \cdot \phi$ (lambda times phi)	Grows with danger, and vanishes at the boundary (because $\phi = 0$ there)

This difference matters enormously, and we will use it repeatedly:

A **constant**-demand filter is asking for something even when the robot is merely touching the edge of the comfort zone. A **proportional** filter asks for nothing at the edge, and only becomes demanding once the robot is well inside.

4. The failure condition

Now we can put it together. The filter needs some choice of joint speeds that satisfies its demand. Substituting our formula for $\dot{\phi}$:

$$L_f \phi + (\text{what motors do}) \leq -\text{demand}$$

The most helpful thing the motors can do is $-c$ (that is what c means: the best possible push downward). So a safe choice exists exactly when

$$L_{f\phi} - C \leq -\text{demand}$$

Rearrange — move everything to one side:

$g = \underbrace{C}_{\substack{\text{braking} \\ \text{power}}} - \underbrace{\text{demand}}_{\substack{\text{what the} \\ \text{filter wants}}} - \underbrace{L_{f\phi}}_{\text{drift}}$
$g \geq 0$ a safe move exists — the filter works
$g < 0$ NO safe move exists — the guarantee is void

We call g the **margin**. It is a subtraction of three things, and it is the heart of everything that follows.

Three observations worth pausing on:

(a) Infeasibility is physics, not a bug. When $g < 0$, the filter has not malfunctioned and the solver has not failed. There is genuinely no admissible movement that meets the requirement. This is a shortfall of physical capability.

(b) The attacker's command does not appear in g . This is subtle and important. The filter chooses its movement by solving “stay as close as possible to what the task wanted, subject to the safety requirement”. The task's wish sits in the “stay close to” part, never in the “subject to” part. So **whether a safe move exists depends only on where the robot IS**, not on what it was asked to do.

The consequence: the attacker cannot craft a devious command. The only lever is **getting the robot to a bad place**. The attack is a navigation problem.

(c) What happens at $g < 0$ depends on the filter's fallback, and there are only two possibilities:

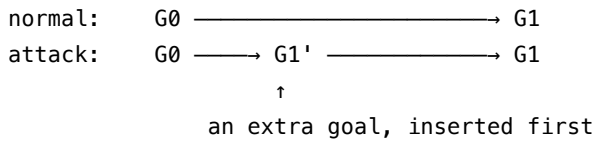
- **Filters with no give (SSA, CBF, SSS).** The requirement cannot be met, so the filter gives up and passes through the original, obstacle-blind command. The robot drives on as if nothing were wrong → **collision**.
- **Filters with give (the “relaxed” and “projected” variants).** They bend the requirement by the smallest amount that makes it satisfiable, stay safe, but stop making progress → **the robot freezes, task never completed**.

So the attacker forces **one of two failures — a crash or a lock-up — without needing to know which**. That is the core result, and it is why an attacker who knows nothing about the filter can still do damage.

5. Two ways to attack

The robot is at a start position (call it **G0**) and has been given a legitimate goal (**G1**). The attacker can issue goals of their own. There are two ways to use that.

Insertion — add a stop along the way

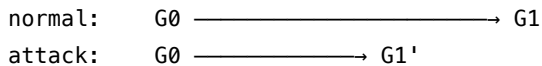


“Also stop by bed 4 on your way.” Perfectly reasonable request.

The trick: **both goals are individually fine**. Going to $G1'$ alone is safe. Going to $G1$ alone is safe. It is the order that breaks the robot — after visiting $G1'$, the arm is in a configuration from which the trip to $G1$ passes through a low-authority basin.

For this to prove anything, we must insist that $G0 \rightarrow G1'$ succeeds **on its own**. Otherwise $G1'$ is just a bad goal, and we have shown nothing about sequencing. SIREN enforces this check on every candidate.

Modification — change the destination



“Actually, put it on the far counter instead.” Also reasonable.

Here there is only one journey, and the attacker chooses its endpoint so that the route passes through a low-authority basin.

How they compare

	Insertion	Modification
What varies	the start of the failing leg	the endpoint of the journey
Extra requirement	$G1'$ must be reachable alone	none
Search difficulty	harder (more constrained)	easier (freer)
Special strength	controls the approach speed and direction	simpler, more direct

That last row matters. By placing $G1'$, the attacker controls how the robot arrives at the danger zone — from which direction and with how much built-up speed. Against filters whose danger number counts approach speed ([section 9](#)), that is a decisive advantage, and insertion is clearly the stronger attack.

6. Four kinds of attacker

Everyone is chasing the same target: **reach a state where $g < 0$** . They differ only in how much of

$$g = C - \text{demand} - L_{\text{f}}\phi$$

they are able to compute.

	Knows the danger formula?	Knows the kind of demand?	Knows the demand's number?
White box	yes	yes	yes
Grey box	yes	yes	no
Weak black box	yes (after one test)	no	no
Strict black box	no	no	no

One thing **everybody** has, in all four cases:

- **The robot's specification.** The mechanical description of a commercial robot — joint layout, speed limits — is published. So u_{lim} and the geometry are known to anyone.
- **A view of the room.** The attacker can see where the obstacles are.
- **A simulator.** They can try things offline before touching the real robot.

That last point deserves emphasis: **all four attackers search in their own simulation**. What differs between the tiers is how faithfully that simulation matches the real robot.

7. White box — you know everything

What you can compute

Everything. You know the danger formula, so you can compute ϕ and the sensitivities, hence c . You know the demand exactly. So you can compute the margin g itself, at every moment of a simulated journey.

The objective

$$\text{score}(\text{candidate goal}) = -(\text{the smallest } g \text{ reached during the journey})$$

The more negative g gets, the better the attack. A goal that drives g below zero is an attack.

Attack steps

1. **Read the configuration.** Which filter, which danger formula, what demand value.
2. **Build the scene.** Robot model from the published specification; obstacle positions from observation.
3. **Propose a candidate goal g_1** inside the allowed workspace and not too close to any obstacle — so it looks like an ordinary request.
4. **Check legitimacy.** For insertion, simulate $g_0 \rightarrow g_1$ alone; discard the candidate unless it succeeds.

5. **Simulate the full journey** ($G_0 \rightarrow G_1' \rightarrow G_1$, or just $G_0 \rightarrow G_1'$) using the real filter, and record g at every step.
6. **Score it** as $-\min g$.
7. **Concentrate the search.** Feed the scores back so the next batch of candidates is drawn near the best ones so far, rather than scattered at random.
8. **Execute the best candidate on the real robot.** Because the simulation matched reality exactly, it should work first time.

Why it is surgical

There is no guesswork anywhere. The attacker computes the exact quantity that governs failure, and aims directly at it.

8. Grey box — you know the kind of filter, not its setting

You know the danger formula and whether the demand is constant or proportional. You do **not** know the actual number (η or λ).

The key insight for constant demand

You cannot compute $g = c - \eta$, because η is unknown. But look at what you are trying to do: find the state where g is smallest. And

$$g = c - \underline{\eta}$$

the SAME unknown number everywhere

Subtracting the same constant from every state **shifts all the scores equally — it does not change which state is worst.** So:

The place where g is smallest is exactly the place where c is smallest. The unknown setting does not affect where to aim, only how far you must go.

This is a genuinely nice result: for constant-demand filters, a grey-box attacker aims at precisely the same target as a white-box attacker. The only thing missing is knowing when the line has been crossed — and for that there is an oracle.

The give-up oracle

When a no-give filter fails, the robot **visibly stops dodging** and drives on as though the obstacle were not there. That is observable from outside. So the attacker does not need to know η : they push toward lower and lower c and **watch for the robot's behaviour to change.** The robot announces its own threshold.

The proportional case is different

If the demand is $\lambda \cdot \phi$, the unknown λ multiplies ϕ , which **varies from state to state.** It does not cancel out. And there is a trap:

at the boundary, $\phi = 0$, so the demand $\lambda \cdot \phi = 0$ too
 $\Rightarrow g = C - 0 = C \geq 0$ always
 $\Rightarrow$ merely touching the edge can NEVER defeat this filter

To beat a proportional filter you must **penetrate**: push far enough inside the comfort zone that $\lambda \cdot \phi$ finally exceeds c . Concretely you need $\phi > c / \lambda$.

So the grey-box objective depends on which kind you face:

constant demand: score = -(smallest C)
proportional demand: score = -(smallest C) + $\beta \times$ (deepest penetration)

where β is a weighting. That second term — the **penetration hedge** — appears whenever the demand might vanish at the boundary.

Attack steps

1. **Identify the family** (constant vs proportional) — this is what “grey” grants you.
2. **Pick a plausible value** for the unknown setting, just so the simulation can run.
3. **Propose candidates** and check legitimacy, as before.
4. **Simulate and score** using c (plus the penetration hedge if proportional) — never using the unknown number.
5. **Concentrate the search** as before.
6. **Try the best candidates on the real robot** and watch for the give-up behaviour.
7. **Optional upgrade**: each real attempt brackets the unknown setting — it failed at this authority level, so the setting is above it; it survived at that level, so it is below. A handful of attempts pins the number down, and you have effectively become a white-box attacker.

9. Weak black box — you know the danger formula is one of two

Now you do not know the demand at all — not even its kind. But you do know the danger formula belongs to one of two families.

The two families

Distance-based (what we saw in section 3):

$$\phi = d_{\min} - d$$

“I get nervous when something is within 10 cm.”

Speed-aware: also counts how fast clearance is shrinking. Writing $\dot{d}$ for “how fast the clearance is changing” (negative when closing in):

$$\phi = d_{\min} - d - k \cdot \dot{d}$$

“I get nervous when something is within 10 cm — **and sooner if it is rushing at me.**” The constant k sets how much the approach speed counts.

Why does the second kind exist at all? If the robot is controlled by forces rather than velocities, it has momentum: commanding a stop does not stop it instantly.

A distance-only danger number would notice trouble too late. Adding the speed term makes it notice earlier. This is not an exotic variant — it is standard, and SPARK's own code will not even let you use the speed-aware formula unless the robot is force-controlled.

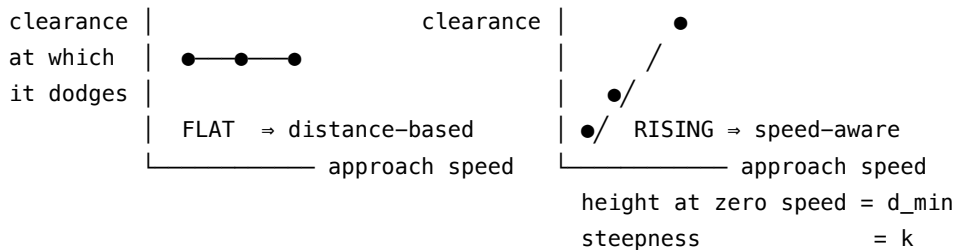
The test that tells them apart

Here is the pleasing part. The two families behave differently in a way you can **watch from outside**: at what distance does the robot start dodging?

distance-based: always starts dodging at the same clearance,
no matter how fast it is approaching

speed-aware: starts dodging **FARTHER OUT** when approaching faster,
because the $-k \cdot d$ term inflates the danger number

So: **drive the arm at the same obstacle two or three times at different speeds, and record the clearance at which it first swerves.**



Two runs classify the family. Three also give you d_{min} and k outright.

Two things make this test attractive:

- **It is harmless.** Nothing crashes, nothing freezes. The robot just does its job while swerving slightly. There is nothing for anyone to detect.
- **It does not need precise instruments.** If you can only spot the swerve once it becomes visually obvious, every measurement is shifted inward by roughly the same amount — which moves the height of the line but not its steepness. And steepness is what classifies the family.

How do you control approach speed if all you can send is goals? Indirectly: a distant goal makes the robot accelerate to full speed; a nearby one keeps it gentle; a preceding goal creates a run-up. **This is another reason insertion is the stronger attack** — it shapes the approach.

A shortcut: the control mode gives the answer away

While building SIREN we found something that makes this even easier in practice. The two danger formulas are **not interchangeable** — each requires a particular kind of robot, and the software enforces it:

Robot is controlled by...	Danger formula it must use
velocity commands (“move this joint at this speed”)	distance-based
force / torque commands (“push this joint this hard”)	speed-aware

The reason is the one from section 3. With velocity control, telling a joint to stop stops it immediately, so clearance responds to the command straight away and distance alone is enough. With force control the robot has momentum, so clearance does not respond immediately — and the formula has to look ahead by counting the approach speed.

Why this matters to an attacker: how a robot is controlled is a published specification, not a secret. So an attacker who knows they are facing a velocity-controlled arm already knows the danger formula is distance-based — and gets weak-black-box knowledge **for free, with no probing at all.**

The speed sweep still earns its keep: it confirms the guess, it measures $d_{\min}$ and k , and it is the fallback when the control mode is not published or the robot switches modes.

What you do with the answer

Once the family is identified you can compute ϕ and c properly. But the demand is still completely unknown, so you must **always** carry the penetration hedge:

$$\text{score} = -(\text{smallest } C) + \beta \times (\text{deepest penetration})$$

Covering the unknown filter: an ensemble

To simulate a journey at all, you need some filter in the loop — and you do not know the real one. The solution is not to guess once, but to **try several and demand that the candidate works against all of them.**

For predicting where the robot goes, the filter’s identity mostly reduces to one thing: **how hard it resists** approaching obstacles. A weak filter lets the robot get close; a strong one makes it swing wide. So the ensemble spans resistance:

```

weak resistance    ┌
medium resistance  ┤ — 3 simulated filters
strong resistance  └

```

Each candidate goal is simulated against all three, scored against each, and then given its **worst** score:

$$\text{final score} = \text{the LOWEST of the three scores}$$

This is deliberately strict. A goal only rates highly if it drives the robot into the low-authority basin **no matter how hard the real filter resists** — so its success does not depend on having guessed correctly.

Attack steps

1. **Run the speed sweep** (2–3 harmless approaches at different speeds).
 2. **Read off the family** from flat-vs-rising, and fit $d_{\min}$ and k .
 3. **Build the 3-filter ensemble** spanning weak/medium/strong resistance.
 4. **Propose candidates** and check legitimacy.
 5. **Simulate each candidate against all three**, score with $-C + \beta \times \text{penetration}$, and keep the **worst** score.
 6. **Concentrate the search** on the best candidates.
 7. **Execute the top-ranked goals on the real robot**, in order, until one lands.
-

10. Strict black box — you know nothing about the filter

The hardest case. You do not know the danger formula, so you cannot compute ϕ . And since the sensitivities are sensitivities of the danger number, at first glance you cannot compute c either — the measuring stick itself is gone.

The rescue

Here is the observation that saves it. **Every collision-safety danger number ever used is built out of clearance.** Nobody invents one from unrelated quantities. So whatever the formula is, it has the shape

$$\phi = (\text{some function of clearance, and possibly its rate})$$

And that means the sensitivities split into two pieces:

how the danger number responds to a joint	=	how the danger number responds to clearance	×	how the clearance responds to that joint
		UNKNOWN (needs ϕ)		KNOWN — pure geometry, from the published robot specification

The unknown piece is **one single number that multiplies everything equally**. It does not depend on which joint. So it **scales** all the sensitivities together — it never changes which joints matter or which poses are bad.

Therefore, define a substitute measured on raw clearance instead of the danger number:

Retreat capacity — how fast the motors can open up clearance. Computable from the published robot specification and the observed obstacles alone. No danger formula, no filter, nothing secret. (Our term, not standard vocabulary.)

And then:

$$\text{true braking power} = (\text{unknown multiplier}) \times \text{retreat capacity}$$

Because the multiplier is common to every pose, **ranking poses by retreat capacity gives the same ordering as ranking by true braking power**. The strict black-box attacker cannot say how bad a pose is in absolute terms — but can say perfectly well which pose is worst. And that is all a search needs.

The slogan: strict black box is not blind, it is unscaled.

(For the simplest and most common danger formula, $\phi = d_{\min} - d$, the unknown multiplier is exactly 1, so the substitute is not even an approximation.)

The objective

$$\text{score} = -(\text{smallest retreat capacity}) + \beta \times (\text{deepest penetration})$$

Same shape as before, with the index-free ruler in place of c .

A bigger ensemble

Two things are unknown now — the resistance and the danger formula family — so the ensemble spans both:

$$\{\text{weak, medium, strong}\} \text{ resistance} \times \{\text{distance-based, speed-aware}\} = 6 \text{ filters}$$

Again scored worst-case: a candidate must work against all six.

A cheap first pass

Six simulations per candidate is expensive. So SIREN first ranks a large pool using a very cheap approximation — the journey with the filter’s resistance turned almost to zero, which needs no optimisation solving — and only spends the full six-way evaluation on the most promising slice.

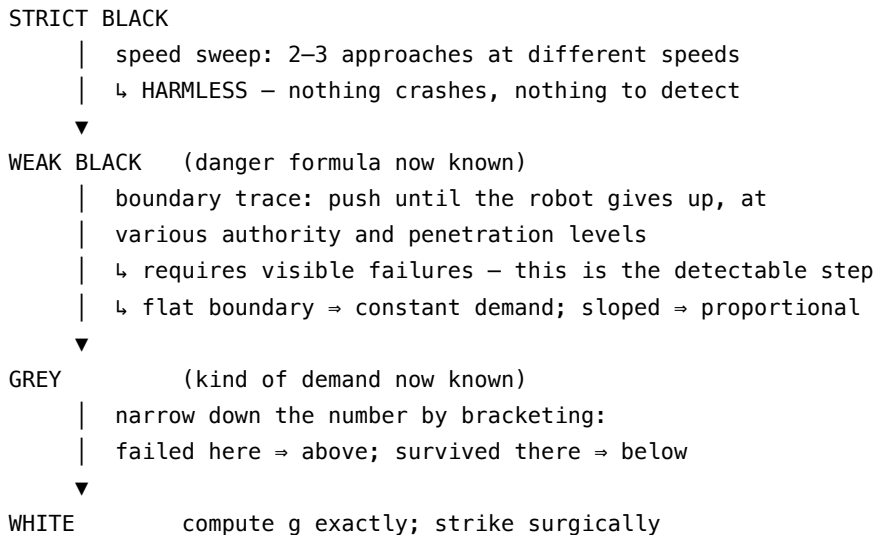
This cheap pass must never be the final judge. The ordinary controller is obstacle-blind, so with the filter effectively switched off the simulated arm ploughs straight through obstacles, and “how deep did it penetrate” stops distinguishing anything. It is a filter for the shortlist, not a verdict.

Attack steps

1. **Build the geometry** from the published robot specification and the observed obstacle layout. Compute retreat capacity across the reachable workspace — no filter, no danger formula needed.
2. **Cheap pass:** propose a large pool of legitimate candidate goals and rank them with the near-zero-resistance approximation.
3. **Full evaluation** of the top slice: simulate each against all six ensemble members.
4. **Score** each with $-\text{retreat capacity} + \beta \times \text{penetration}$, and take the **worst** across the six.
5. **Concentrate the search** on the winners and repeat.
6. **Execute the top-ranked goals on the real robot**, in order.
7. **Optional but recommended:** run the speed sweep from [section 9](#) first. It is harmless, costs two or three ordinary-looking commands, and promotes you to weak black box — halving the ensemble and sharpening the ruler.

11. The ladder: climbing from black to white

The four tiers are not separate scenarios. They are **rungs**, and an attacker can climb them. The currency is attempts on the real robot.



Three consequences worth stating plainly:

Most of the climb is invisible. Only the middle rung needs the robot to actually fail. Identifying the danger formula — the hardest unknown — is done with ordinary-looking commands that produce no incident at all.

The climb can happen off-site. These robots are sold commercially. An attacker can buy the same model and identify the danger formula and the filter family in their own workshop, at zero risk, then arrive at the target already knowing most of what matters. Only the site's specific setting remains.

The number of real attempts is a natural measurement. It orders the tiers cleanly — white ≈ one attempt, grey a few, black more — and it doubles as a stealth measure, since every attempt is an opportunity to be noticed.

12. Summary

The physical fact. A robot's ability to push danger away — its braking power c — depends on its pose and the obstacle layout, and can be small. Safety filters demand a certain rate of danger reduction. When the demand exceeds what the motors can deliver, the margin

$$g = C - \text{demand} - \text{drift}$$

goes negative and **no safe action exists**. This is physics, not a software defect, and it is not fixable by better code.

Why it is a security problem. The attacker never breaks any rule. They issue legitimate goals — an extra stop, a different destination — chosen so the robot's own goal-seeking behaviour carries it into a low-authority basin. State readings stay truthful, every command is valid, and the filter solves exactly the problem it was designed to solve.

Why it works without inside knowledge. The target region is defined mostly by geometry, which is public. The unknown parts of g either cancel out (a constant demand shifts every state equally), or reduce to a common multiplier that changes no rankings (an unknown danger formula), or can be hedged against (penetration covers the proportional case). What cannot be reasoned away is covered by simulating against an ensemble of plausible filters and demanding the candidate beat all of them.

Why the outcome is bad either way. At $g < 0$ the filter must either give up and pass through the unsafe command — a **collision** — or bend its requirement and stop making progress — a **lock-up**. Tightening the filter to avoid one makes the other more likely. There is no setting that avoids both.

The four attackers at a glance

	What it aims at	Ensemble	Hedge?	Real attempts
White	the exact margin g	1	no	≈ 1
Grey (constant)	smallest c — same target as white	1	no	a few
Grey (proportional)	smallest c , deepest penetration	1	yes	a few
Weak black	smallest c , deepest penetration	3	yes	more
Strict black	smallest retreat capacity , deepest penetration	6	yes	most

Reading down that column tells the whole degradation story: the exact certificate $g \rightarrow$ the part of it that survives an unknown setting (c) $\rightarrow$ the part that survives an unknown danger formula (retreat capacity), with the penetration hedge appearing exactly at the point where the demand's kind stops being known.

Appendix A — The mathematics in full

Sections 3 and 4 explained what each quantity means. This appendix derives all of them properly: where ϕ , ϕ , $L_f \phi$, $L_g \phi$, C , the demand and the margin g actually come from, and how each is computed from the robot model.

Nothing here is needed to follow the main text — it is the reference version.

A.0 Notation, and one clash to watch

Symbol	Meaning	Size
x	robot state	n
u	control input (what we command)	m
$q, \dot{q}$	joint positions, joint velocities	—
$f(x)$	drift vector field — motion with zero command	n
$G(x)$	control matrix — how commands enter the dynamics	$n \times m$
$d(x)$	clearance: robot-to-obstacle distance	scalar
$\phi(x)$	safety index (“danger number”)	scalar per pair
$L_f \phi$	drift term of ϕ	scalar
$L_g \phi$	control-sensitivity row of ϕ	$1 \times m$
u_{lim}	per-joint command limit	m
C	control authority	scalar
η, λ	demand parameters	scalar
g	the margin $C - \text{demand} - L_f \phi$	scalar
μ	exact multi-constraint margin (an LP value)	scalar

Clash warning. The letter g is doing two jobs, and this is inherited from the literature rather than chosen: $L_g \phi$ uses g for the control matrix (standard Lie-derivative notation), while g alone is our margin. To keep them apart, this appendix writes the control matrix as $G(x)$ and keeps the conventional name $L_g \phi$ for the Lie derivative along it.

A.1 The system model

Every derivation below assumes the system is **control-affine** — the command enters linearly:

$$\dot{x} = \underbrace{f(x)}_{\text{drift}} + \underbrace{G(x) \cdot u}_{\text{control}}$$

This is not a restriction in practice; robot models take this form. Two cases matter here, and they are exactly the $\mathfrak{d1}/\mathfrak{d2}$ split in SPARK’s benchmarks.

Velocity-controlled (kinematic, SPARK $\mathfrak{d1}$). The command is the joint velocity, $u = \dot{q}$, and the state is just the configuration, $x = q$:

$$\dot{x} = \dot{q} = u \quad \Rightarrow \quad f(x) = 0, \quad G(x) = I$$

There is no drift: switch the motors off and nothing moves. **This is why the drift term vanishes in our experiments.**

Torque-controlled (dynamic, SPARK $\mathfrak{d2}$). The command is joint torque, $u = \tau$, and the state carries velocity too, $x = (q, \dot{q})$. With the standard manipulator equation $M(q) \ddot{q} + C(q, \dot{q}) \dot{q} + G_{\text{grav}}(q) = \tau$:

$$\dot{x} = \underbrace{\begin{bmatrix} \dot{q} \\ -M^{-1} (C(q, \dot{q}) \dot{q} + G_{\text{grav}}(q)) \end{bmatrix}}_{f(x)} + \underbrace{\begin{bmatrix} 0 \\ M^{-1} \end{bmatrix}}_{G(x)} u$$

Now $f(x) \neq 0$: momentum and gravity keep the robot moving with no command. That non-zero drift is what makes the “unavoidable growth” corner of A.7 reachable.

A.2 The safety index ϕ

Start from the physical requirement — keep at least d_{min} of clearance:

$$\phi_0(x) = d_{min} - d(x) \quad \phi_0 > 0 \iff \text{closer than allowed}$$

Relative degree. How many times must we differentiate ϕ_0 before the command u appears? Differentiating once:

$$\dot{\phi}_0 = (\partial\phi_0/\partial x) \cdot (f(x) + G(x)u)$$

The command shows up only through $(\partial\phi_0/\partial x) \cdot G(x)$.

- **Velocity control:** $G = I$, so $(\partial\phi_0/\partial x) \cdot G = \partial\phi_0/\partial \dot{q} \neq 0$ — the command appears immediately. **Relative degree 1**, and ϕ_0 can be used directly.
- **Torque control:** $\partial\phi_0/\partial x$ has no $\dot{q}$ component (clearance depends on position only), and $G = [0; M^{-1}]$ is non-zero only in the $\dot{q}$ block — so the product is **zero**. The command does not appear. **Relative degree 2**, and ϕ_0 is unusable as a constraint: no torque has any first-order effect on it.

The fix (this is what SSA is for). Add derivative terms until the relative degree is 1 again. The general n -th order index is

$$\phi = \phi_0 + \sum_{i=1..n} k_i \cdot \phi_0^{(i)}$$

with $\phi_0^{(i)}$ the i -th time derivative and the k_i chosen so the characteristic polynomial $1 + k_1 s + \dots + k_n s^n$ has negative real roots (no overshoot). For relative degree 2, one term suffices:

$$\phi = \phi_0 + k \dot{\phi}_0 = (d_{min} - d) + k(-\dot{d}) = d_{min} - d - k \cdot \dot{d}$$

Approaching means $\dot{d} < 0$, so $-k \cdot \dot{d} > 0$ and ϕ is inflated by the approach speed — the robot becomes “nervous” earlier when closing fast. This is exactly the distance-vs-speed-aware distinction of section 9, and it now has a reason: **the speed term is not a design flourish, it is what restores relative degree 1 under torque control.**

(SPARK ships a nonlinear variant of the same idea, $\phi = d_{min}^n - d^n + k \cdot (\text{closing rate})$; setting $n = 1$ recovers the linear form. It also asserts at construction that the first-order index requires a $D1$ robot and the second-order one a $D2$ robot — the relative-degree argument enforced in code.)

A.3 Where $L_f \phi$ and $L_g \phi$ come from

Apply the chain rule to $\phi(x)$ and substitute the dynamics:

$$\begin{aligned} \dot{\phi} &= d\phi/dt = (\partial\phi/\partial x) \cdot \dot{x} \\ &= (\partial\phi/\partial x) \cdot (f(x) + G(x)u) \\ &= \underbrace{(\partial\phi/\partial x) \cdot f(x)}_{\text{define } L_f \phi} + \underbrace{(\partial\phi/\partial x) \cdot G(x)}_{\text{define } L_g \phi} \cdot u \end{aligned}$$

So the two symbols are simply **names for the two halves of one chain rule**:

$$\begin{aligned}
L_f \phi &:= (\partial\phi/\partial x) \cdot f(x) && \text{a scalar — the drift of the danger number} \\
L_g \phi &:= (\partial\phi/\partial x) \cdot G(x) && \text{a } 1 \times m \text{ row — one sensitivity per joint} \\
\Rightarrow \quad \dot{\phi} &= L_f \phi + L_g \phi \cdot u && (A.1)
\end{aligned}$$

These are **Lie derivatives** — the rate of change of a scalar field along a vector field. That is all the name means; no extra machinery is implied.

Sanity check for the kinematic case: $f = 0$ and $G = I$ give $L_f \phi = 0$ and $L_g \phi = \partial\phi/\partial q$, so $\dot{\phi} = (\partial\phi/\partial q) \cdot \dot{q}$. Correct.

A.4 The sensitivity in closed form

$L_g \phi$ is not abstract — for collision constraints it has an explicit formula.

Let $p(q)$ be the robot point nearest the obstacle and o the obstacle point. Then $d = ||p(q) - o||$, and differentiating a norm gives

$$\partial d / \partial q = n^T \cdot J_p(q), \quad n = (p - o) / ||p - o||$$

where n is the **unit normal** (the direction from obstacle to robot) and $J_p = \partial p / \partial q$ is the **point Jacobian** of that robot point. So

$$\partial d / \partial q = n^T J_p \quad \text{"how clearance responds to each joint"}$$

For the first-order index $\phi = d_{\min} - d$ under velocity control:

$$L_g \phi = \partial\phi/\partial q = -n^T J_p \quad (A.2)$$

Read (A.2) as the geometric statement it is: the sensitivity is the robot's Jacobian projected onto the obstacle direction. A joint matters only insofar as moving it displaces the endangered point **along the line to the obstacle**. Motion perpendicular to that line contributes nothing. This is why authority collapses in awkward poses — not because the joints are slow, but because their motion no longer projects onto the escape direction.

For the second-order index under torque control, $\partial\phi/\partial\dot{q} = -k \cdot (\partial d / \partial q)$ and $G = [0; M^{-1}]$, so

$$L_g \phi = (\partial\phi/\partial\dot{q}) \cdot M^{-1} = -k \cdot n^T J_p M^{-1} \quad (A.3)$$

Same geometric direction $n^T J_p$, now scaled by k and mapped through the inverse inertia. **The direction is governed by the distance gradient in both cases** — the fact A.10 depends on.

A.5 Control authority c , derived

How fast can the actuators drive ϕ down, at best? The commands live in a box

$$U = \{ u : |u_k| \leq u_{\lim,k}, \quad k = 1..m \}$$

and we want the most negative achievable value of the control term in (A.1):

$$C := \max_{\{u \in U\}} (-L_g \phi \cdot u) = \max_{\{u \in U\}} \sum_k (-[L_g \phi]_k) u_k$$

The sum decouples: each u_k appears in exactly one term and the box constrains each independently. Maximise term by term with

$$u_{k*} = u_{\lim,k} \cdot \text{sign}(-[L_g \phi]_k)$$

which contributes $u_{\text{lim},k} \cdot |[L_g \phi]_k|$. Summing:

$$C = \sum_k u_{\text{lim},k} \cdot |[L_g \phi]_k| \quad (\text{A.4})$$

The absolute value is not a modelling choice — it falls out of the box being **symmetric**: a joint that raises danger forwards lowers it backwards, so either way it contributes its full magnitude.

(Equivalently $C = ||\text{diag}(u_{\text{lim}}) \cdot L_g \phi^T||_1$, the support function of the box — the L1 norm being dual to the L-inf box. Worth knowing only if you want to swap in a different actuator set: replace the support function and everything downstream still holds.)

Immediate consequence. If $L_g \phi = 0$ then $C = 0$: the command has no first-order effect on this constraint, and no filter can help. That is the exact degenerate case relative degree was invented to avoid (A.2), and near-degenerate versions of it — $n^T J_p$ almost zero — are precisely the low-authority basins the attack targets.

A.6 The demand

A value-based filter does not merely want ϕ to fall; it fixes a **rate**:

$$\dot{\phi} \leq -\text{demand}(\phi)$$

Two forms cover the whole family:

constant	demand = η	SSA, r-SSA, p-SSA
proportional	demand = $\lambda \cdot \phi$	CBF, SSS and relaxed variants

The difference is entirely about behaviour **at the boundary** $\phi = 0$:

constant:	demand(0) = $\eta > 0$	bites even when merely grazing
proportional:	demand(0) = 0	asks nothing at the boundary

which is the formal reason grazing can defeat an η -filter but never a $\lambda\phi$ -filter, and hence why every attacker that cannot rule out a proportional demand must carry a penetration term (section 8).

A.7 The margin g , and the feasibility theorem

Claim. A command satisfying the filter's requirement exists **iff** $g \geq 0$, where $g := C - \text{demand} - L_f \phi$.

Proof. A satisfying command exists exactly when the best achievable rate still meets the requirement:

$$\begin{aligned}
 & \exists u \in U : \dot{\phi} \leq -\text{demand} \\
 \Leftrightarrow & \min_{\{u \in U\}} (L_f \phi + L_g \phi \cdot u) \leq -\text{demand} \\
 \Leftrightarrow & L_f \phi + \min_{\{u \in U\}} (L_g \phi \cdot u) \leq -\text{demand} \\
 \Leftrightarrow & L_f \phi - C \leq -\text{demand} \quad [\text{by (A.4)}] \\
 \Leftrightarrow & C - \text{demand} - L_f \phi \geq 0 \\
 \Leftrightarrow & g \geq 0 \quad [] \quad (\text{A.5})
 \end{aligned}$$

SIREN computes the cheap certificate $\min_i g_i$ every step and the LP on demand (exact_margin_lp), since the LP costs a solve per timestep.

A.9 Why the index-free ruler works

Strict black-box does not know ϕ , and c is defined through ϕ — so at first sight the ruler is lost. It is not, and here is the reason in full.

Every collision index is a function of clearance (and possibly its rate): $\phi = F(d)$ for the position-based family. Then by the chain rule

$$\partial\phi/\partial q = F'(d) \cdot \partial d/\partial q \quad \Rightarrow \quad L_g \phi = F'(d) \cdot L_g d$$

Substituting into (A.4), the scalar $F'(d)$ factors straight out of every term:

$$\begin{aligned} C_\phi &= \sum_k u_{\text{lim},k} |F'(d) \cdot [L_g d]_k| \\ &= |F'(d)| \cdot \sum_k u_{\text{lim},k} |[L_g d]_k| \\ &= |F'(d)| \cdot C_d \end{aligned} \tag{A.6}$$

where C_d — the **retreat capacity** — uses raw clearance and needs no index at all, only the published robot model.

The unknown index rescales; it never redirects. Hence:

$$\begin{array}{lll} \phi = d_{\min} - d & F' = -1 & C_\phi = C_d \quad \text{exactly} \\ \phi = d_{\min}^n - d^n & F' = -n d^{n-1} & C_\phi = n d^{n-1} C_d \end{array}$$

For the linear index the substitute is not even an approximation. For a nonlinear one the factor varies with d but is monotone in it, so the **ordering of poses** by authority is preserved up to that reweighting. Ranking survives; absolute scale does not — the precise sense in which strict black-box is unscaled rather than blind.

For the velocity-augmented index the same argument runs through (A.3): the factor is k and the map is M^{-1} , and the direction is still $n^\top J_p$.

A.10 The attack objectives, in symbols

Every tier maximises a score over candidate goals G' ; they differ only in which terms of g are computable. Writing $x_t(G')$ for the state trajectory induced by pursuing G' , and $(\cdot)_+ = \max(0, \cdot)$:

$$\begin{array}{ll} \text{white} & J = - \min_t g(x_t) \\ & = - \min_t [C_\phi - \text{demand} - L_f \phi] \quad (\text{all known}) \\ \\ \text{gray, } \eta & J = - \min_t [C_\phi - L_f \phi] \\ & \eta \text{ is a constant offset} \Rightarrow \text{shifts } J \text{ uniformly} \Rightarrow \text{SAME argmax} \\ & \text{as white. Only the THRESHOLD is unknown, and the robot's} \\ & \text{own give-up supplies it.} \\ \\ \text{gray, } \lambda\phi & J = - \min_t C_\phi + \beta \cdot \max_t (\phi)_+ \\ & \lambda \text{ multiplies the state-varying } \phi, \text{ so it does NOT factor} \\ & \text{out; and demand}(0)=0 \Rightarrow \text{grazing can never work} \Rightarrow \text{the} \\ & \text{penetration term is mandatory, needing } \phi > C/\lambda. \end{array}$$

weak black $J = -\min_t C_\phi + \beta \cdot \max_t (\phi)_+$
 index known (so C_ϕ available) but demand shape unknown
 $\Rightarrow$ hedge regardless.

strict black $J = -\min_t C_d + \beta \cdot \max_t (\text{penetration})$
 index unknown $\Rightarrow$ fall back to (A.6)'s index-free ruler.

ensemble $J = \min_k J_k$ over surrogate filters k (worst case)

A.11 Where each quantity is computed

Quantity	Formula	Code
$\phi, L_g \phi, L_f \phi$	(A.1)–(A.3)	read from SPARK's index — world/sim/probe.py
C	(A.4)	derived.control_authority
demand	A.6	derived.demand_vector
g , forced slack	(A.5)	derived.margin
μ (exact LP)	A.8	derived.exact_margin_lp
objectives	A.10	search/threat.py

A.12 What this appendix does not establish

Stated plainly, since the experiment tested it:

The derivations above show $g < 0$ is a **correct certificate** that the guarantee is void — that part holds. They do **not** show that g is a useful search signal for choosing goals, and measurement says it is not: across 60 candidates on one scene, $\min_t g$ separated successful from unsuccessful attacks by 0.0004 and varied by only 0.035 in total. The authority landscape is flat over the admissible goal region, so ranking candidates by g performs no better than chance. A correct certificate and a useful objective are different things, and only the first is established here.

This document accompanies SIREN. The maths of sections 3–4 and Appendix A lives in *world/derived.py*; the four attackers live in *search/threat.py*.